```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from matminer.datasets import load_dataset

df_k = load_dataset("matbench_log_kvrh")
df_g = load_dataset("matbench_log_gvrh")

print("Bulk modulus dataset:", df_k.shape)
print("Shear modulus dataset:", df_g.shape)
```

```
Bulk modulus dataset: (10987, 2)
Shear modulus dataset: (10987, 2)
```

```python
df = df_k.copy()

df = df.rename(columns={"log10(K_VRH)": "target"})

print(df.head())
```

```
                                           structure    target
0  [[0. 0. 0.] Ca, [1.37728887 1.57871271 3.73949...  1.707570
1  [[3.14048493 1.09300401 1.64101398] Mg, [0.625...  1.633468
2  [[ 2.06884519  2.40627241 -0.45891585] Si, [1....  1.908485
3  [[2.06428082 0.         2.06428082] Pd, [0.   ...  2.117271
4  [[3.09635262 1.0689416  1.53602403] Mg, [0.593...  1.690196
```

```python
#feature engineering from crystal structures
feature_rows = []

for structure in df["structure"]:
    lattice = structure.lattice

    feature_rows.append({
        "density": structure.density,
        "volume": structure.volume,
        "num_sites": len(structure),

        "a": lattice.a,
        "b": lattice.b,
        "c": lattice.c,

        "alpha": lattice.alpha,
        "beta": lattice.beta,
        "gamma": lattice.gamma,

        "abc_mean": np.mean([lattice.a, lattice.b, lattice.c]),
        "abc_std": np.std([lattice.a, lattice.b, lattice.c]),

        "vpa": structure.volume / len(structure),
```

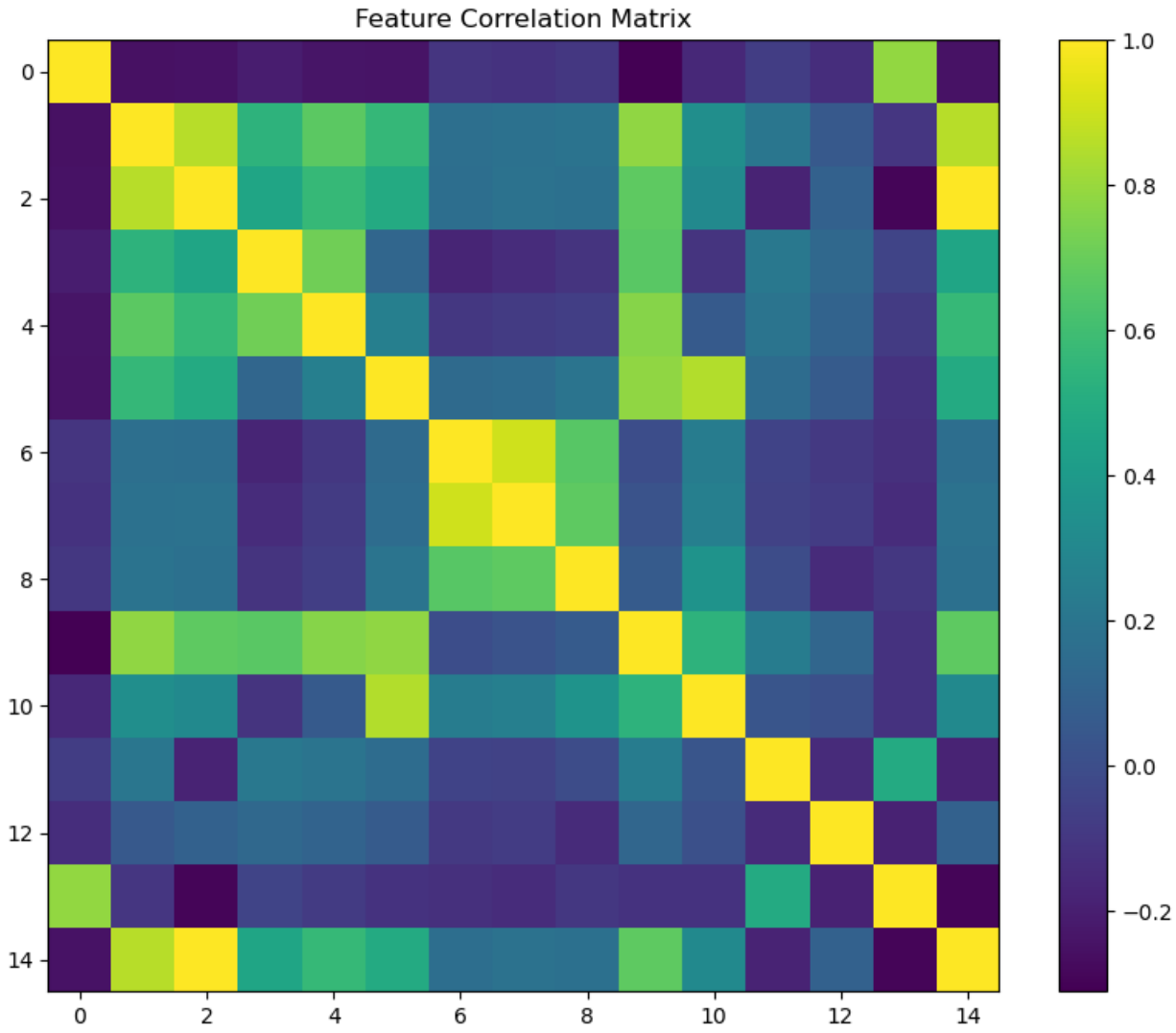
```
        "num_elements": len(structure.composition.elements),

        "avg_atomic_weight": structure.composition.weight /
len(structure),

        "formula_units": structure.composition.num_atoms
    })

X = pd.DataFrame(feature_rows)
y = df["target"]

print("Feature matrix:", X.shape)
X.head()
```

```
Feature matrix: (10987, 15)
```

```
     density      volume  num_sites         a         b         c
alpha  \
0   6.317495  105.426749          5  4.377179  4.377179  6.313292
110.283388
1   4.339424  126.744646          5  4.700520  4.700520  6.629742
110.762959
2   5.149289   87.354159          5  4.189935  4.189935  5.791120
111.208024
3  11.367986   70.371420          4  4.128562  4.128562  4.128562
90.000000
4   3.337111  120.472408          5  4.667972  4.667972  6.439149
111.251847

          beta  gamma  abc_mean   abc_std        vpa  num_elements  \
0  110.283388   90.0  5.022550  0.912693  21.085350             3
1  110.762959   90.0  5.343594  0.909444  25.348929             3
2  111.208024   90.0  4.723663  0.754806  17.470832             3
3   90.000000   90.0  4.128562  0.000000  17.592855             2
4  111.251848   90.0  5.258364  0.834941  24.094482             3

   avg_atomic_weight  formula_units
0           80.21888            5.0
1           66.24340            5.0
2           54.17660            5.0
3          120.44000            4.0
4           48.42160            5.0
```

```
#feature engineering/descriptor extraction from crystal structure
dataset
corr = X.corr()

plt.figure(figsize=(10,8))
plt.imshow(corr)
plt.colorbar()
```

```
plt.title("Feature Correlation Matrix")
plt.show()
```

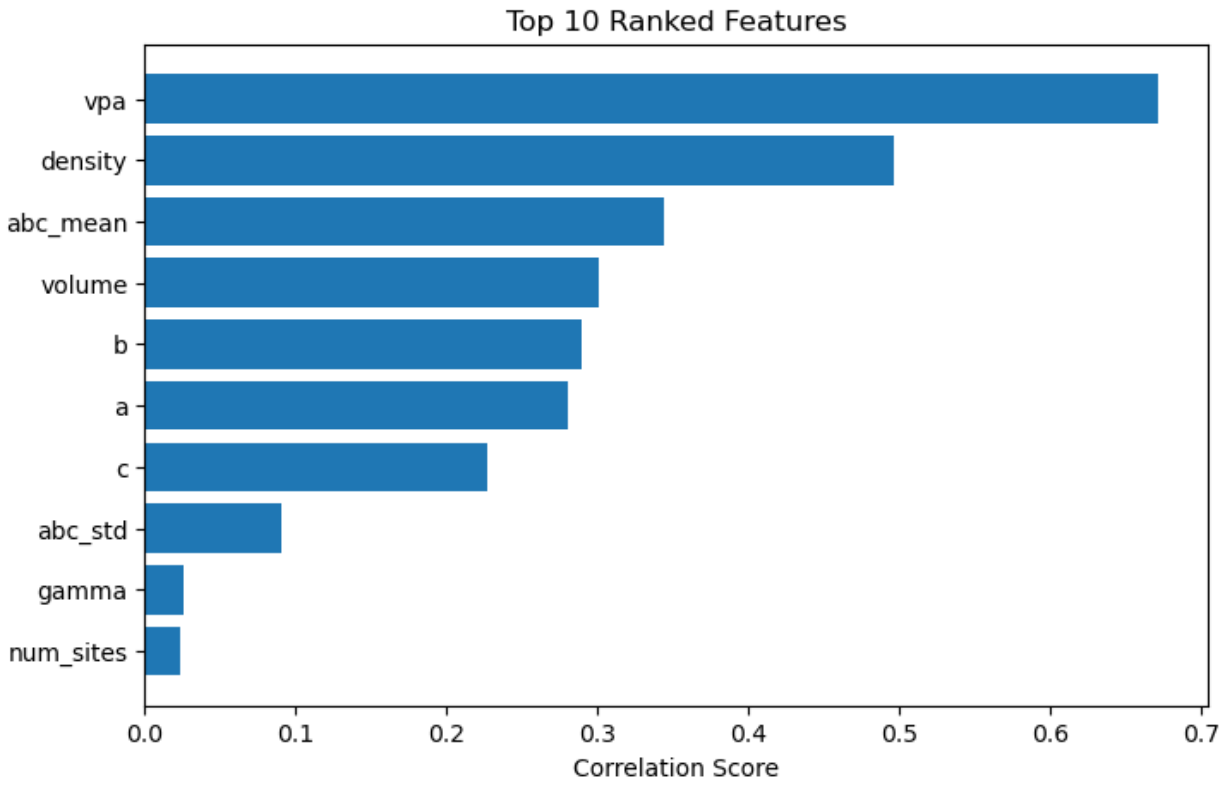



```
#this cell measures how closely each feature is related to the target variable.
feature_scores = {}

for col in X.columns:
    feature_scores[col] = abs(X[col].corr(y))

feature_ranking = pd.DataFrame({
    "feature": feature_scores.keys(),
    "score": feature_scores.values()
}).sort_values("score", ascending=False)
```

```
print(feature_ranking)
```

```
               feature     score
11                 vpa  0.671004
0              density  0.496281
9             abc_mean  0.343893
1               volume  0.300500
4                    b  0.289988
3                    a  0.280256
5                    c  0.226928
10             abc_std  0.090524
8                gamma  0.026078
2            num_sites  0.024174
14       formula_units  0.024174
13  avg_atomic_weight  0.022410
7                 beta  0.022186
6                alpha  0.012728
12        num_elements  0.010612
```

```
#this cell plots top 10 most important features.
top10 = feature_ranking.head(10)

plt.figure(figsize=(8,5))
plt.barh(top10["feature"], top10["score"])
plt.gca().invert_yaxis()
plt.xlabel("Correlation Score")
plt.title("Top 10 Ranked Features")
plt.show()
```




```
#select  top 10 most imp features from complete feature matrix(X).
selected_features = feature_ranking.head(10)["feature"].tolist()

X_selected = X[selected_features]

print(X_selected.shape)
print(selected_features)
```

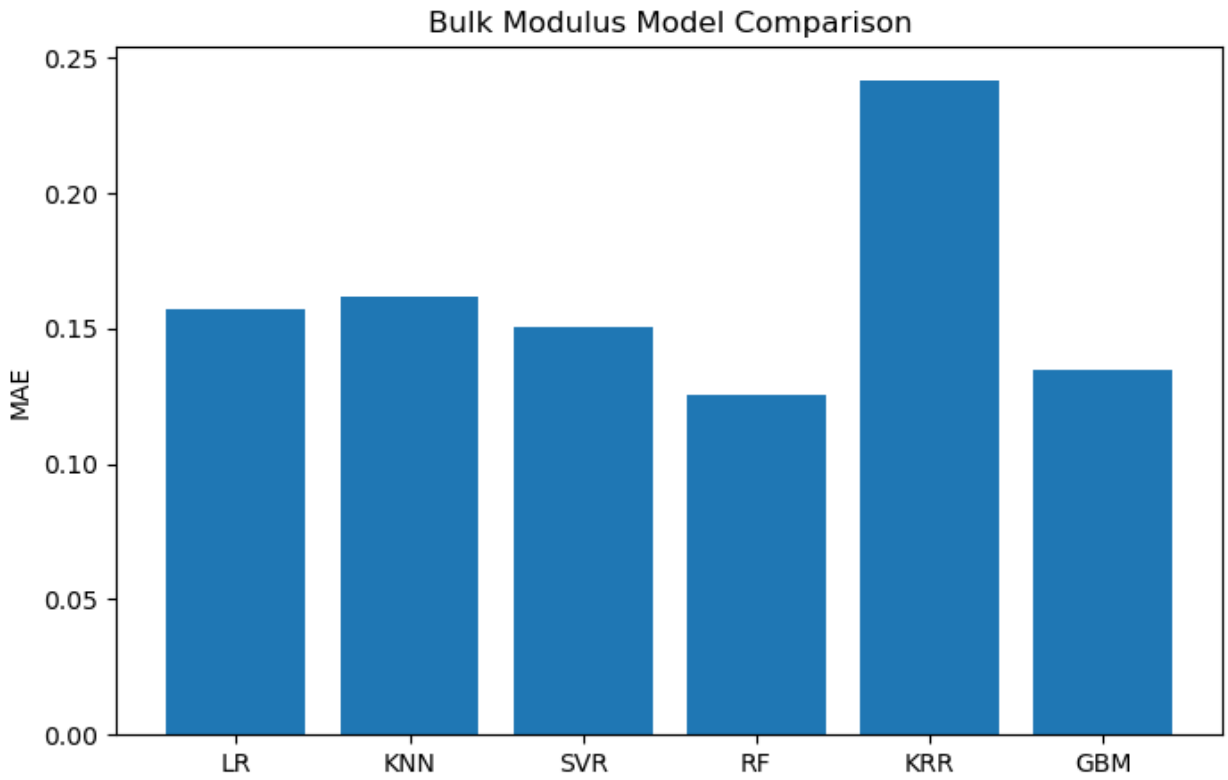
```
(10987, 10)
['vpa', 'density', 'abc_mean', 'volume', 'b', 'a', 'c', 'abc_std',
'gamma', 'num_sites']
```

```
#select top features to train six different regression models.
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics import mean_absolute_error

X_train, X_test, y_train, y_test = train_test_split(
    X_selected,
    y,
    test_size=0.2,
```

```
    random_state=42
)

models = {
    "LR": LinearRegression(),
    "KNN": KNeighborsRegressor(),
    "SVR": SVR(),
    "RF": RandomForestRegressor(random_state=42),
    "KRR": KernelRidge(),
    "GBM": GradientBoostingRegressor(random_state=42)
}

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, pred)
    results[name] = mae

print(results)
```

```
{'LR': 0.15712749381649987, 'KNN': 0.16151594148407675, 'SVR':
0.15086715860628067, 'RF': 0.12582791914118371, 'KRR':
0.2419145898713808, 'GBM': 0.1350097152180481}
```

```
#this is for the plot comparision of diff results obtained by training
the models.
plt.figure(figsize=(8,5))
plt.bar(results.keys(), results.values())
plt.ylabel("MAE")
plt.title("Bulk Modulus Model Comparison")
plt.show()
```

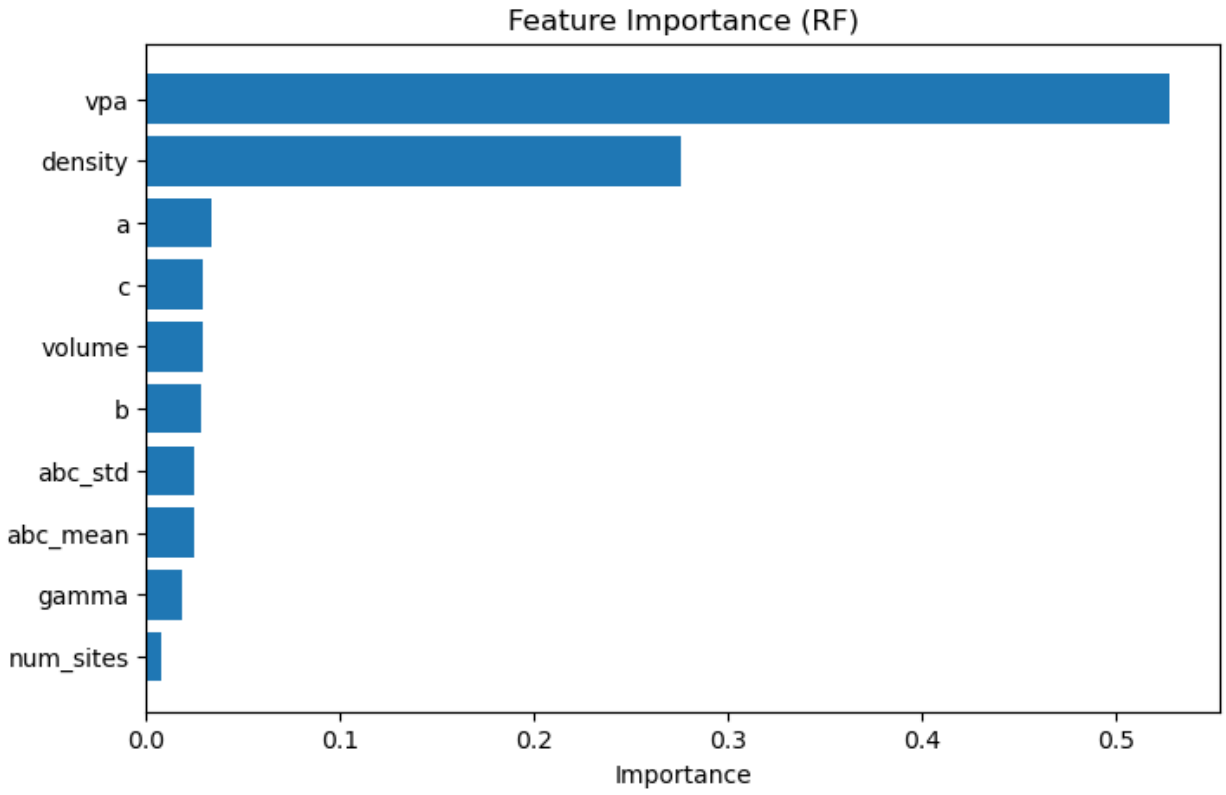



```
#obtaining the top features for the best regression model trained.
best_model = models["RF"]

importance_df = pd.DataFrame({
    "feature": X_selected.columns,
    "importance": best_model.feature_importances_
}).sort_values("importance", ascending=False)

print(importance_df)
```

```
     feature  importance
0        vpa    0.527473
1    density    0.275672
5          a    0.034099
6          c    0.028980
3     volume    0.028889
4          b    0.028660
7    abc_std    0.024587
2   abc_mean    0.024561
8      gamma    0.018825
9  num_sites    0.008253
```

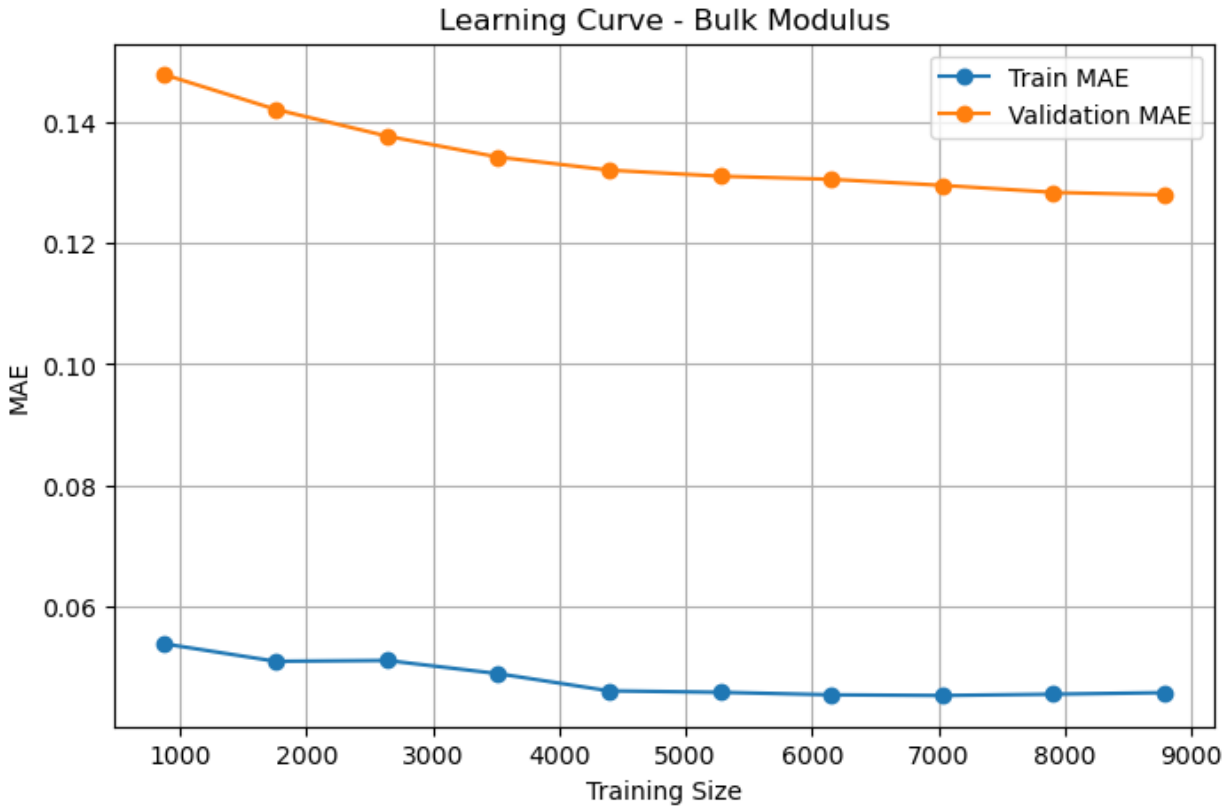
```
#plot for the feature importance
plt.figure(figsize=(8,5))
plt.barh(importance_df["feature"], importance_df["importance"])
plt.gca().invert_yaxis()
```

```
plt.xlabel("Importance")
plt.title("Feature Importance (RF)")
plt.show()
```




```
#this cell helps us compare the MAE for train and validation points , so that we can find the prediction accuracy/reliability of the model
from sklearn.model_selection import learning_curve

train_sizes, train_scores, test_scores = learning_curve(
    RandomForestRegressor(random_state=42),
    X_selected,
    y,
    cv=5,
    scoring="neg_mean_absolute_error",
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)

train_mae = -train_scores.mean(axis=1)
test_mae = -test_scores.mean(axis=1)

plt.figure(figsize=(8,5))
plt.plot(train_sizes, train_mae, marker="o", label="Train MAE")
plt.plot(train_sizes, test_mae, marker="o", label="Validation MAE")
plt.xlabel("Training Size")
```

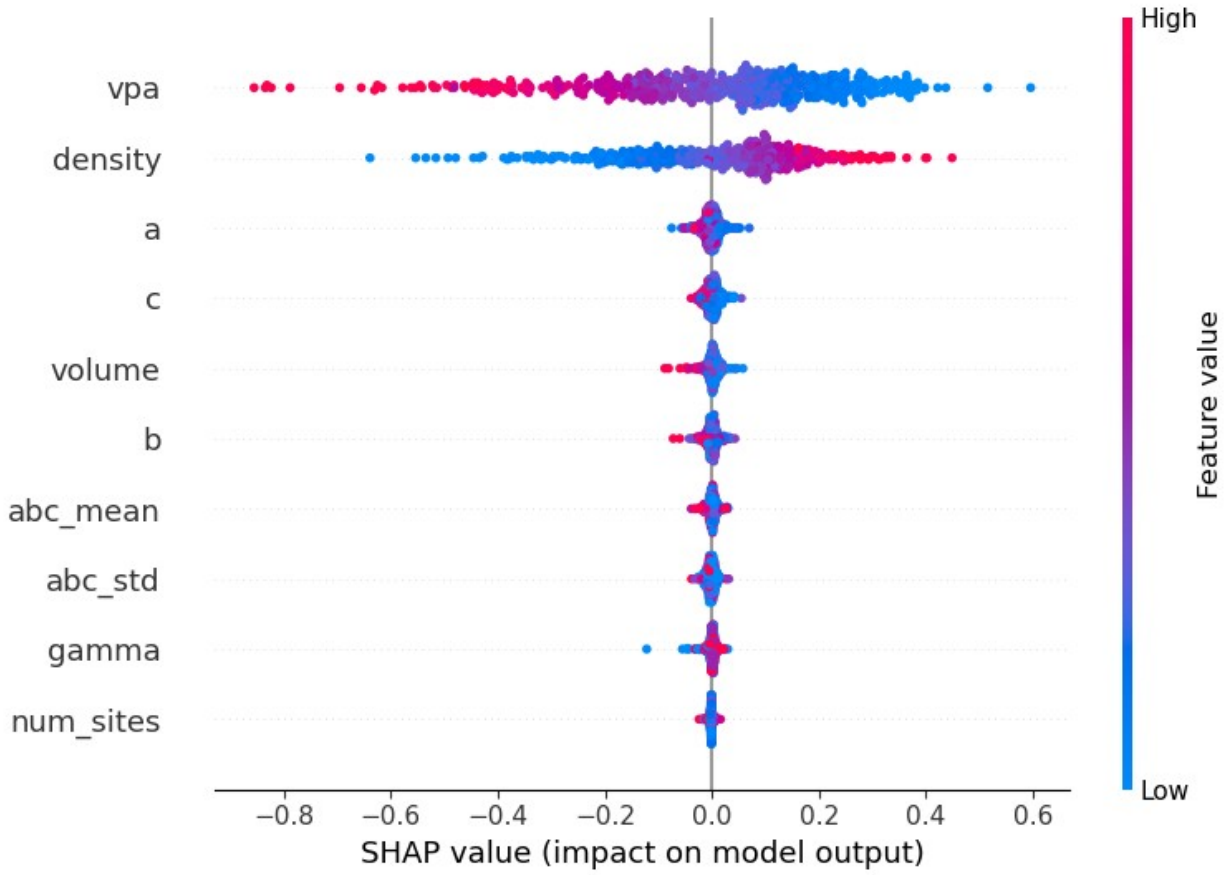
```
plt.ylabel("MAE")
plt.title("Learning Curve - Bulk Modulus")
plt.legend()
plt.grid(True)
plt.show()
```




```
!pip install shap

Requirement already satisfied: shap in c:\users\aspire go 14\
anaconda3\lib\site-packages (0.51.0)
Requirement already satisfied: numpy>=2 in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (2.3.5)
Requirement already satisfied: scipy in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (1.17.1)
Requirement already satisfied: scikit-learn in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (1.7.2)
Requirement already satisfied: pandas in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (2.3.3)
Requirement already satisfied: tqdm>=4.27.0 in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in c:\users\aspire go
14\anaconda3\lib\site-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (0.0.8)
```

```
Requirement already satisfied: numba in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (0.62.1)
Requirement already satisfied: llvmlite in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (0.45.1)
Requirement already satisfied: cloudpickle in c:\users\aspire go 14\
anaconda3\lib\site-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in c:\users\aspire go
14\anaconda3\lib\site-packages (from shap) (4.15.0)
Requirement already satisfied: colorama in c:\users\aspire go 14\
anaconda3\lib\site-packages (from tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\
aspire go 14\anaconda3\lib\site-packages (from pandas->shap)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\aspire go 14\
anaconda3\lib\site-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\aspire go
14\anaconda3\lib\site-packages (from pandas->shap) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\aspire go 14\
anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas-
>shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in c:\users\aspire go 14\
anaconda3\lib\site-packages (from scikit-learn->shap) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\aspire
go 14\anaconda3\lib\site-packages (from scikit-learn->shap) (3.5.0)
```

```
#this cell show cases the SHAP , and here the result is reproduced
same as in the research paper.
import shap

sample_X = X_selected.sample(500, random_state=42)

explainer = shap.TreeExplainer(models["RF"])

shap_values = explainer.shap_values(sample_X)

shap.summary_plot(shap_values, sample_X)
```




```
#repeating the same loading process for the shear modulus
df = df_g.copy()

df = df.rename(columns={"log10(G_VRH)": "target"})

print(df.head())
```

```
                                           structure    target
0  [[0. 0. 0.] Ca, [1.37728887 1.57871271 3.73949...  1.447158
1  [[3.14048493 1.09300401 1.64101398] Mg, [0.625...  1.518514
2  [[ 2.06884519  2.40627241 -0.45891585] Si, [1....  1.740363
3  [[2.06428082 0.         2.06428082] Pd, [0.   ...  1.707570
4  [[3.09635262 1.0689416  1.53602403] Mg, [0.593...  1.602060
```

```
#again we are selecting features for shear modulus as we did before
feature_rows = []

for structure in df["structure"]:
    lattice = structure.lattice

    feature_rows.append({
        "density": structure.density,
        "volume": structure.volume,
```

```
        "num_sites": len(structure),

        "a": lattice.a,
        "b": lattice.b,
        "c": lattice.c,

        "alpha": lattice.alpha,
        "beta": lattice.beta,
        "gamma": lattice.gamma,

        "abc_mean": np.mean([lattice.a, lattice.b, lattice.c]),
        "abc_std": np.std([lattice.a, lattice.b, lattice.c]),

        "vpa": structure.volume / len(structure),

        "num_elements": len(structure.composition.elements),

        "avg_atomic_weight": structure.composition.weight /
len(structure),

        "formula_units": structure.composition.num_atoms
    })

X_g = pd.DataFrame(feature_rows)
y_g = df["target"]

print(X_g.shape)
```

```
(10987, 15)
```

```
#the total feautres and selecting top one
feature_scores_g = {}

for col in X_g.columns:
    feature_scores_g[col] = abs(X_g[col].corr(y_g))

feature_ranking_g = pd.DataFrame({
    "feature": feature_scores_g.keys(),
    "score": feature_scores_g.values()
}).sort_values("score", ascending=False)

print(feature_ranking_g)
```

```
               feature     score
11                 vpa  0.674377
0              density  0.337290
9             abc_mean  0.282555
3                    a  0.269510
4                    b  0.262741
1               volume  0.256945
5                    c  0.153062
```

```
13  avg_atomic_weight  0.117785
6                alpha  0.045066
8                gamma  0.033450
7                 beta  0.032833
2           num_sites  0.025569
14      formula_units  0.025569
10            abc_std  0.017672
12       num_elements  0.000974
```

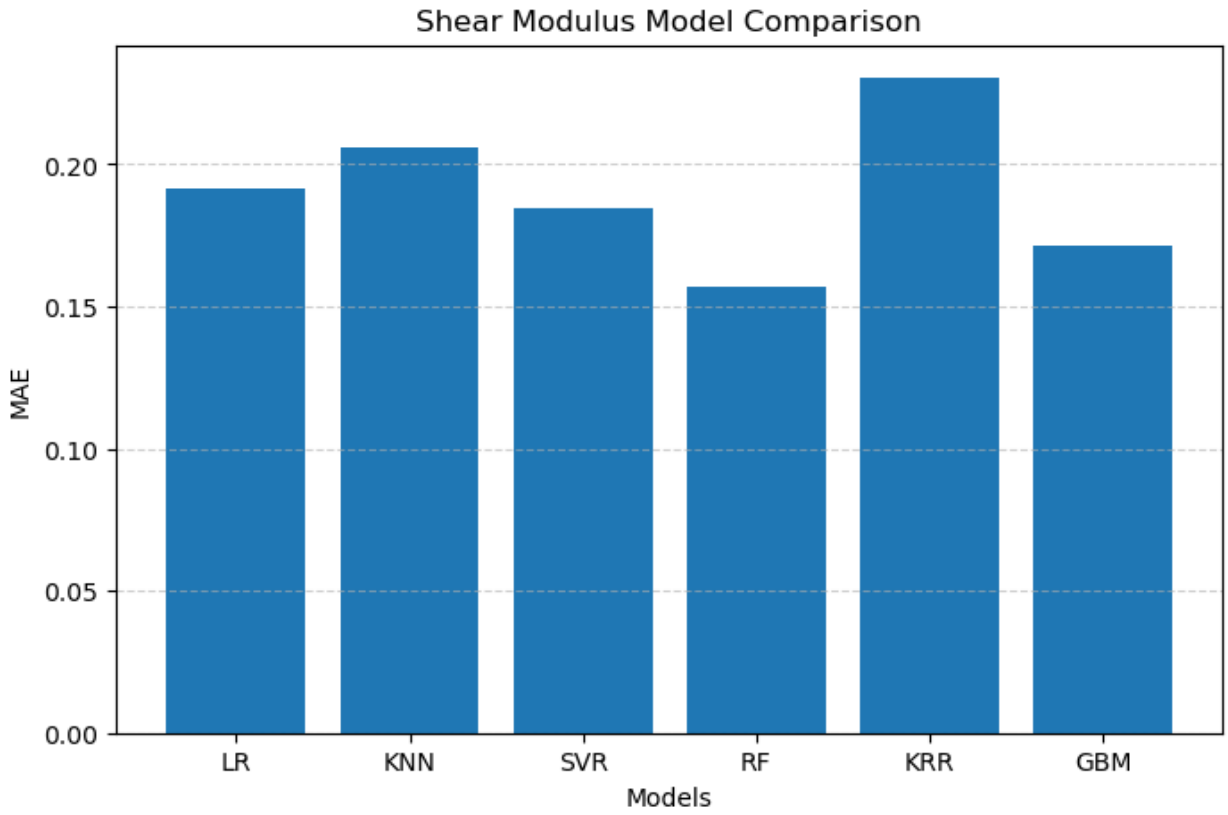
```
selected_features_g = feature_ranking_g.head(10)["feature"].tolist()

X_selected_g = X_g[selected_features_g]

print(X_selected_g.shape)
print(selected_features_g)
```

```
(10987, 10)
['vpa', 'density', 'abc_mean', 'a', 'b', 'volume', 'c',
'avg_atomic_weight', 'alpha', 'gamma']
```

```
#dividing the dataset in test and training split
#training 6 diff regression models and finding the best performing one
X_train_g, X_test_g, y_train_g, y_test_g = train_test_split(
    X_selected_g,
    y_g,
    test_size=0.2,
    random_state=42
)

models_g = {
    "LR": LinearRegression(),
    "KNN": KNeighborsRegressor(),
    "SVR": SVR(),
    "RF": RandomForestRegressor(random_state=42),
    "KRR": KernelRidge(),
    "GBM": GradientBoostingRegressor(random_state=42)
}

results_g = {}

for name, model in models_g.items():
    model.fit(X_train_g, y_train_g)
    pred = model.predict(X_test_g)
    mae = mean_absolute_error(y_test_g, pred)
    results_g[name] = mae

print(results_g)
```

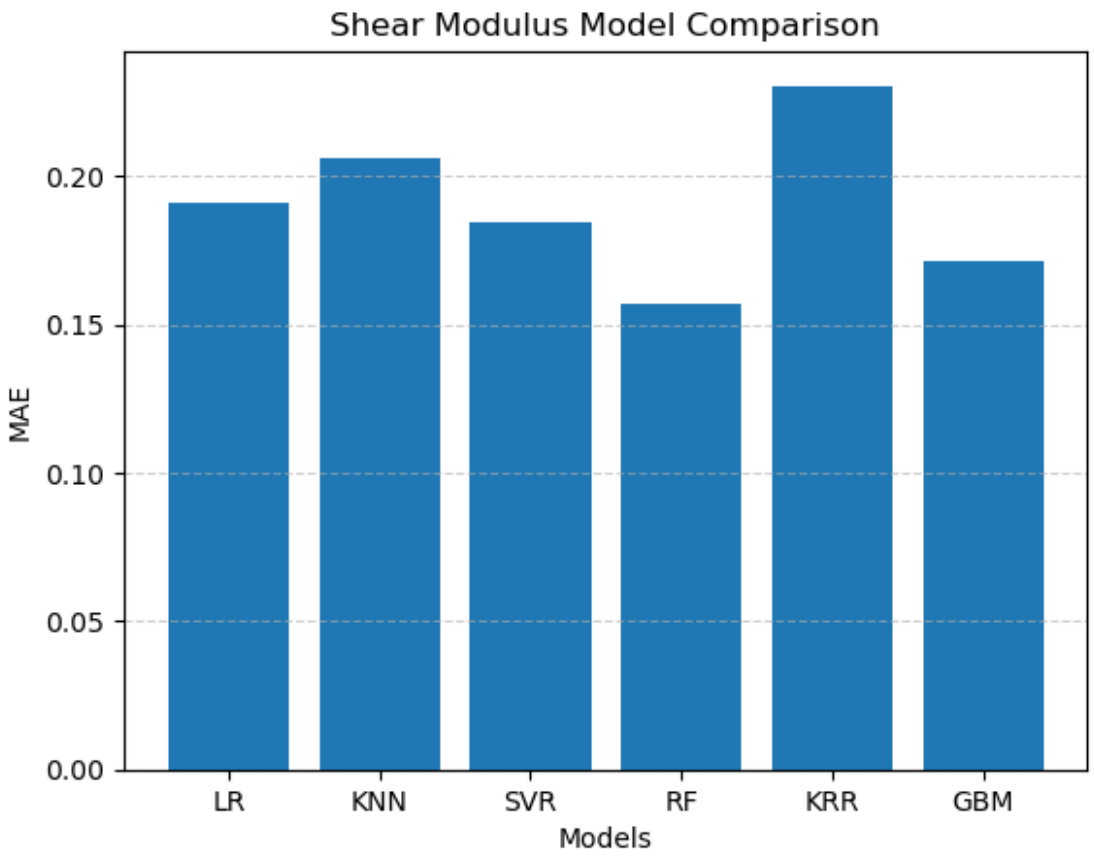
```
{'LR': 0.1912598777094534, 'KNN': 0.2060378925694474, 'SVR':
0.18452504102277825, 'RF': 0.15665998837835785, 'KRR':
0.23043641332553821, 'GBM': 0.17152196936386832}
```

```
#plots for validating the results found in the last cell
plt.figure(figsize=(8,5))
plt.bar(results_g.keys(), results_g.values())

plt.ylabel("MAE")
plt.xlabel("Models")
plt.title("Shear Modulus Model Comparison")
plt.grid(axis="y", linestyle="--", alpha=0.6)

plt.show()
```



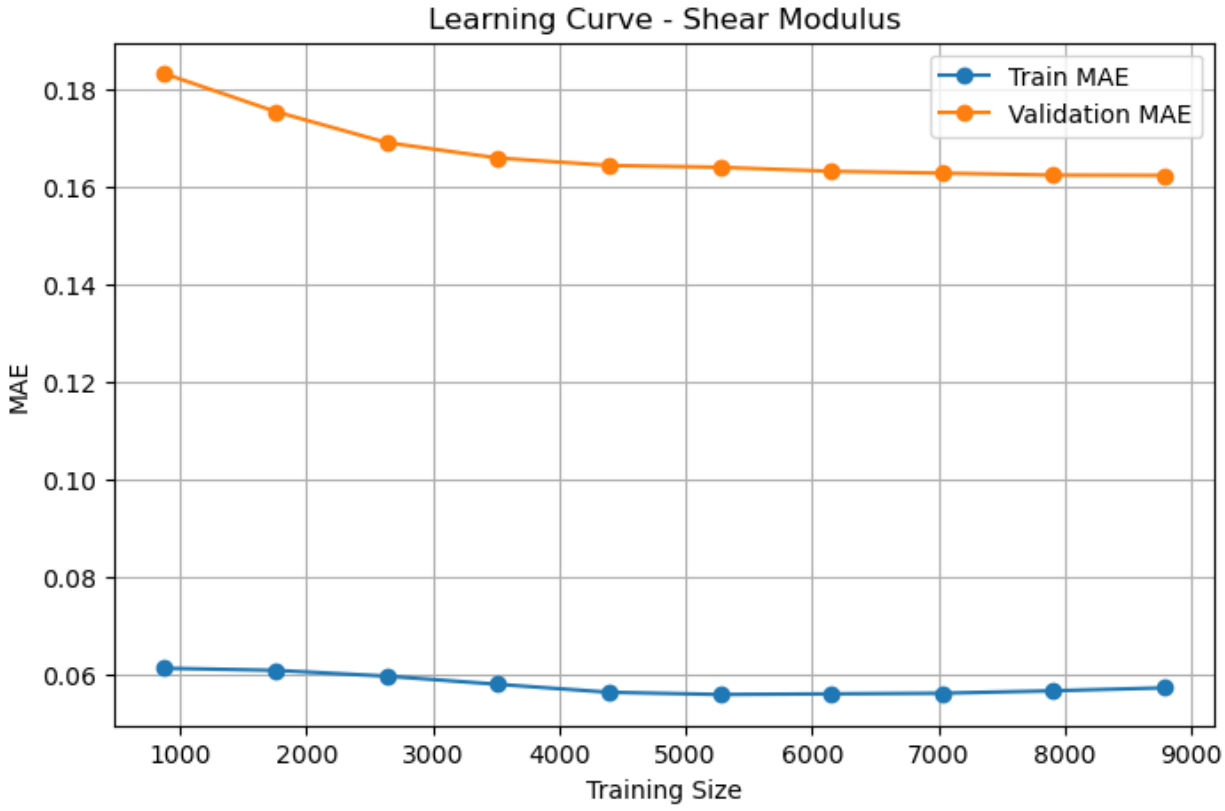

```
#comparing the results through the plots
model_names = list(results_g.keys())
mae_values = list(results_g.values())

bars = plt.bar(model_names, mae_values)

best_idx = mae_values.index(min(mae_values))
bars[best_idx].set_linewidth(2)

plt.ylabel("MAE")
plt.xlabel("Models")
plt.title("Shear Modulus Model Comparison")
plt.grid(axis="y", linestyle="--", alpha=0.6)
```

```
plt.show()
```




```
#using the learning curve for the comparision of MAE values.
train_sizes_g, train_scores_g, test_scores_g = learning_curve(
    RandomForestRegressor(random_state=42),
    X_selected_g,
    y_g,
    cv=5,
    scoring="neg_mean_absolute_error",
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)

train_mae_g = -train_scores_g.mean(axis=1)
test_mae_g = -test_scores_g.mean(axis=1)

plt.figure(figsize=(8,5))
plt.plot(train_sizes_g, train_mae_g, marker="o", label="Train MAE")
plt.plot(train_sizes_g, test_mae_g, marker="o", label="Validation
MAE")

plt.xlabel("Training Size")
```

```
plt.ylabel("MAE")
plt.title("Learning Curve - Shear Modulus")
plt.legend()
plt.grid(True)

plt.show()
```

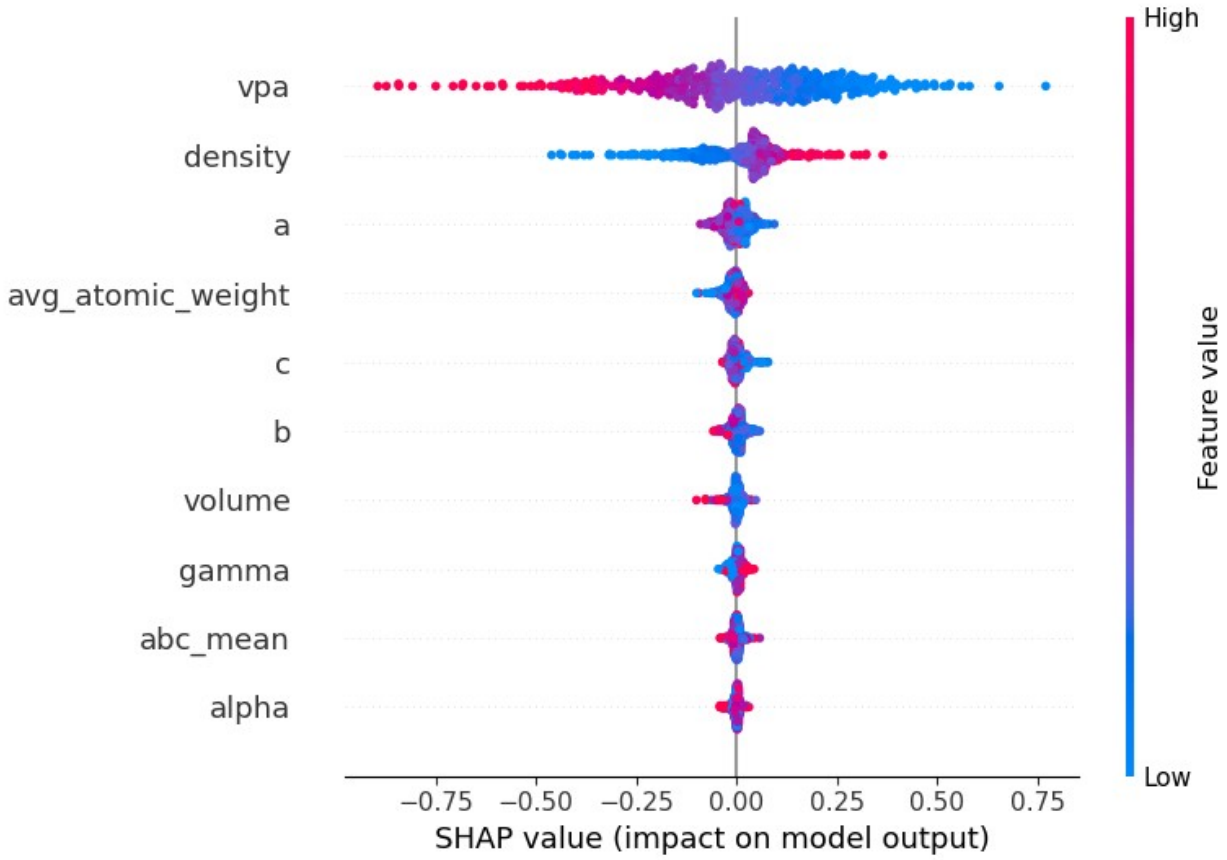



```
# recreate selected shear features
selected_features_g = feature_ranking_g.head(10)["feature"].tolist()

X_selected_g = X_g[selected_features_g]

print(X_selected_g.shape)
print(selected_features_g)
```

```
(10987, 10)
['vpa', 'density', 'abc_mean', 'a', 'b', 'volume', 'c',
'avg_atomic_weight', 'alpha', 'gamma']
```

```
#using the SHAP for final reproduction of the results produced in the
paper
import shap

sample_X_g = X_selected_g.sample(500, random_state=42)
```

```
explainer_g = shap.TreeExplainer(models_g["RF"])

shap_values_g = explainer_g.shap_values(sample_X_g)

shap.summary_plot(shap_values_g, sample_X_g)
```




```
import matplotlib.pyplot as plt

# Create figure
plt.figure(figsize=(12,5))

# Bulk modulus histogram
plt.subplot(1, 2, 1)
plt.hist(df_k["log10(K_VRH)"], bins=30)
plt.xlabel("log10(K_VRH)")
plt.ylabel("Frequency")
plt.title("Distribution of Bulk Modulus")

# Shear modulus histogram
plt.subplot(1, 2, 2)
plt.hist(df_g["log10(G_VRH)"], bins=30)
plt.xlabel("log10(G_VRH)")
plt.ylabel("Frequency")
```

```
plt.title("Distribution of Shear Modulus")

plt.tight_layout()
plt.show()
```